

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG

GOOD DESIGN SPECIFICATION
HỆ THỐNG ĐI CHỢ TIỆN LỢI -
NAVIMART

Môn học: Phát triển phần mềm theo chuẩn kỹ năng ITSS
Giảng viên hướng dẫn: ThS. Nguyễn Mạnh Tuấn
Nhóm thực hiện: Nhóm 5

DANH SÁCH THÀNH VIÊN

STT	Họ và tên	MSSV
1	Trịnh Thành An	20235633
2	Nguyễn Mạnh Đức	20235681
3	Đỗ Danh Dũng	20235685
4	Đinh Xuân Thái Dương	20235689
5	Nguyễn Hồng Dương	20235693

Mục lục

1	Thiết kế mô-đun (Modularity)	2
1.1	Khái niệm cơ bản	2
1.2	Tầm quan trọng của tính mô-đun	2
1.3	Đặc điểm của thiết kế tốt và lý tưởng của phần mềm	2
2	Coupling và Cohesion	3
2.1	Coupling	3
2.2	Cohesion	3
3	Các nguyên tắc thiết kế SOLID	4
4	Áp dụng đánh giá tính mô-đun vào hệ thống NaviMart	4
4.1	Cấp độ 1: Phân rã theo Miền Nghiệp Vụ (Domain-Driven Modularity) . .	4
4.2	Cấp độ 2: Phân rã theo Vai Trò (Role-Based Modularity / Mô hình BCE)	5
4.3	Cấp độ 3: Phân rã theo Trách Nhiệm (Functional Modularity)	5
5	Đánh giá Coupling và Cohesion chi tiết từng Module của NaviMart	6
5.1	Đánh giá tổng quan hệ thống	6
5.2	Module 1: Quản lý Xác thực và Người dùng (Auth Module)	6
5.3	Module 2: Quản lý Nhóm Gia đình (Family Module)	6
5.4	Module 3: Lập danh sách mua sắm (Shopping Lists Module)	6
5.5	Module 4: Quản lý Kho Thực phẩm (Pantry Module)	7
5.6	Module 5: Kế hoạch ăn uống (Meals Module)	7
5.7	Module 6: Công thức & Trợ lý ảo (Recipes & AI Chef Module)	7
5.8	Module 7: Thống kê & Báo cáo (Statistics Module)	7
5.9	Module 8: Quản trị hệ thống (Admin Module)	8
6	Đánh giá nguyên lý SOLID chi tiết từng Module của NaviMart	8
6.1	Đánh giá tổng quan hệ thống	8
6.2	Module 1: Quản lý Xác thực và Người dùng (Auth Module)	8
6.3	Module 2: Quản lý Nhóm Gia đình (Family Module)	9
6.4	Module 3: Lập danh sách mua sắm (Shopping Lists Module)	9
6.5	Module 4: Quản lý Kho Thực phẩm (Pantry Module)	10
6.6	Module 5: Kế hoạch ăn uống (Meals Module)	10
6.7	Module 6: Công thức & Trợ lý ảo (Recipes & AI Chef Module)	11
6.8	Module 7: Thống kê & Báo cáo (Statistics Module)	11
6.9	Module 8: Quản trị hệ thống (Admin Module)	12
7	Kết luận	12

1 Thiết kế mô-đun (Modularity)

1.1 Khái niệm cơ bản

Thiết kế mô-đun (Modularity) là kỹ thuật chia nhỏ hệ thống phần mềm lớn, phức tạp thành các thành phần độc lập, nhỏ hơn gọi là mô-đun. Mỗi mô-đun đảm nhận một tập hợp các chức năng cụ thể và được đóng gói một cách độc lập. Các mô-đun này liên kết và tương tác với nhau qua các giao diện lập trình (interfaces) được định nghĩa rõ ràng để xây dựng nên một hệ thống hoàn chỉnh.

1.2 Tầm quan trọng của tính mô-đun

Tính mô-đun đóng vai trò nền tảng trong phát triển phần mềm hiện đại:

- **Giảm thiểu độ phức tạp:** Chia nhỏ hệ thống lớn thành các phần độc lập dễ quản lý, áp dụng triệt để nguyên lý “chia để trị” (divide and conquer).
- **Tăng khả năng tái sử dụng (Reusability):** Một mô-đun thiết kế tốt có thể tái sử dụng dễ dàng ở nhiều phân hệ của hệ thống hoặc trong các dự án khác mà không cần chỉnh sửa mã nguồn.
- **Nâng cao tính dễ bảo trì (Maintainability):** Khi có lỗi hoặc cần thay đổi nghiệp vụ, lập trình viên chỉ cần cô lập và sửa đổi bên trong mô-đun liên quan, giảm thiểu rủi ro ảnh hưởng dây chuyền (regression bugs) đến các mô-đun khác.
- **Phát triển song song hiệu quả:** Cho phép nhiều lập trình viên hoặc các nhóm nhỏ phát triển song song các mô-đun riêng biệt cùng một lúc mà không bị xung đột mã nguồn.
- **Dễ dàng kiểm thử (Testability):** Các bài kiểm thử đơn vị (Unit Tests) có thể được viết riêng cho từng mô-đun để kiểm chứng tính đúng đắn trước khi tích hợp hệ thống.

1.3 Đặc điểm của thiết kế tốt và lý tưởng của phần mềm

Một thiết kế cấu trúc phần mềm được coi là tốt và lý tưởng khi nó đạt được hai chỉ số cốt lõi:

- **Liên kết lỏng lẻo (Loose Coupling):** Sự phụ thuộc và liên kết giữa các mô-đun khác nhau được giảm thiểu tối đa. Các mô-đun tương tác qua lại qua các tham số tối giản hoặc DTO, không chia sẻ tài nguyên hay vùng nhớ trực tiếp.
- **Gắn kết chặt chẽ (Tight Cohesion):** Mọi thành phần và dòng mã bên trong một mô-đun hoạt động đồng bộ và tập trung cao độ để thực hiện một trách nhiệm nghiệp vụ cụ thể, rõ ràng và duy nhất.

2 Coupling và Cohesion

2.1 Coupling

Coupling là thước đo mức độ phụ thuộc lẫn nhau giữa các mô-đun trong hệ thống phần mềm. Độ liên kết càng cao (tight coupling) thì hệ thống càng dễ bị lỗi dây chuyền và khó bảo trì. Thiết kế tốt luôn hướng tới độ liên kết thấp (loose coupling).

Các cấp độ của Coupling từ tệ nhất (chặt chẽ) đến tốt nhất (lỏng lẻo):

1. **Content Coupling (Chặt chẽ nhất / Tệ nhất):** Một mô-đun trực tiếp can thiệp, sửa đổi hoặc sử dụng các biến, logic nội bộ được đóng gói của mô-đun khác.
2. **Common Coupling:** Các mô-đun chia sẻ chung vùng dữ liệu toàn cục (Global Variables) hoặc cơ sở dữ liệu dùng chung không qua kiểm soát.
3. **Control Coupling:** Mô-đun này điều khiển luồng thực thi của mô-đun khác bằng cách truyền vào các tham số điều khiển (ví dụ: các biến flag).
4. **Stamp Coupling:** Các mô-đun truyền cho nhau cấu trúc dữ liệu phức tạp (Object/Struct) mặc dù mô-đun nhận chỉ sử dụng một vài trường thông tin cụ thể bên trong.
5. **Data Coupling (Lỏng lẻo nhất / Tốt nhất):** Các mô-đun giao tiếp thuần túy qua các tham số dữ liệu nguyên thủy hoặc các DTO tối giản, chia sẻ vừa đủ thông tin cần thiết.

2.2 Cohesion

Cohesion là thước đo mức độ liên kết chặt chẽ và tập trung vào một nhiệm vụ duy nhất của các thành phần bên trong một mô-đun. Độ gắn kết càng cao (tight cohesion) chứng tỏ thiết kế mô-đun càng tốt và chuyên biệt.

Các cấp độ của Cohesion từ tệ nhất (thấp) đến tốt nhất (cao):

1. **Coincidental Cohesion (Thấp nhất / Tệ nhất):** Các thành phần được gom lại trong cùng một mô-đun hoàn toàn ngẫu nhiên và không có mối quan hệ nghiệp vụ hay logic nào.
2. **Logical Cohesion:** Các thành phần thực hiện các tác vụ tương tự nhau về mặt logic nhưng có nghiệp vụ hoàn toàn khác nhau.
3. **Temporal Cohesion:** Các thành phần được gom chung do chúng phải thực thi vào cùng một thời điểm (ví dụ: cấu hình khởi tạo hệ thống).
4. **Procedural Cohesion:** Các thành phần được thực thi theo một trình tự hoặc quy trình nghiệp vụ cố định.
5. **Communicational Cohesion:** Các thành phần hoạt động trên cùng một dữ liệu đầu vào hoặc đầu ra.
6. **Sequential Cohesion:** Kết quả đầu ra của một thành phần bên trong mô-đun được sử dụng làm đầu vào trực tiếp cho thành phần tiếp theo.
7. **Functional Cohesion (Cao nhất / Tốt nhất):** Tất cả các thành phần trong mô-đun cùng phối hợp chặt chẽ để thực hiện một chức năng đơn lẻ, tập trung nghiệp vụ và duy nhất.

3 Các nguyên tắc thiết kế SOLID

Nguyên tắc SOLID là bộ 5 quy tắc thiết kế hướng đối tượng giúp mã nguồn trở nên linh hoạt, dễ đọc, dễ mở rộng và dễ bảo trì:

- **S - Single Responsibility Principle (SRP):** Một lớp hoặc mô-đun chỉ nên đảm nhận một trách nhiệm nghiệp vụ duy nhất, tức là chỉ có duy nhất một lý do để thay đổi.
- **O - Open-Closed Principle (OCP):** Các thành phần phần mềm nên mở cho việc mở rộng (thêm chức năng mới) nhưng đóng đối với việc sửa đổi mã nguồn hiện tại.
- **L - Liskov Substitution Principle (LSP):** Các đối tượng của lớp con phải có khả năng thay thế hoàn toàn cho các đối tượng của lớp cha mà không làm thay đổi tính đúng đắn của chương trình.
- **I - Interface Segregation Principle (ISP):** Thay vì tạo giao diện (interface) chung công kênh chứa quá nhiều phương thức, nên tách thành nhiều interface nhỏ, chuyên biệt để client không bị ép buộc phụ thuộc vào các phương thức không dùng tới.
- **D - Dependency Inversion Principle (DIP):** Các mô-đun cấp cao không nên phụ thuộc trực tiếp vào các mô-đun cấp thấp; cả hai nên phụ thuộc vào các lớp trừu tượng (interfaces/abstractions). Abstraction không được phụ thuộc vào chi tiết cụ thể, mà chi tiết phải phụ thuộc vào abstraction.

4 Áp dụng đánh giá tính mô-đun vào hệ thống NaviMart

4.1 Cấp độ 1: Phân rã theo Miền Nghiệp Vụ (Domain-Driven Modularity)

Kiến trúc backend của hệ thống NaviMart (phát triển trên nền tảng NestJS) được tổ chức phân rã một cách khoa học thành 8 phân hệ nghiệp vụ độc lập, tự quản lý phạm vi của mình:

1. **Auth Module (Xác thực):** Quản lý tài khoản, đăng nhập, đăng ký, mã hóa mật khẩu và kiểm soát phân quyền qua mã bảo mật JWT.
2. **Family Module (Nhóm Gia đình):** Xử lý kết nối các thành viên trong gia đình, liên kết các tài khoản dùng chung và phân quyền nội bộ nhóm.
3. **Pantry Module (Tủ lạnh):** Theo dõi danh sách thực phẩm hiện có trong tủ lạnh gia đình, quản lý số lượng và tự động tính toán hạn sử dụng (HSD).
4. **Shopping Lists Module (Mua sắm):** Lập kế hoạch mua sắm, tạo danh sách hàng cần mua và đồng bộ tiến độ đi chợ giữa các thành viên.
5. **Meals Module (Kế hoạch ăn uống):** Quản lý lịch trình ăn uống (Meal Plan) và các phiên nấu nướng (Cooking Sessions).

6. **Recipes Module (Công thức & AI Chef):** Quản lý dữ liệu công thức nấu ăn, gợi ý món ăn thông minh dựa trên lượng thực phẩm sẵn có trong Pantry, tích hợp dịch vụ trợ lý ảo AI Chef.
7. **Statistics Module (Thống kê):** Phân tích dữ liệu tiêu dùng, báo cáo chi tiêu mua sắm và thống kê xu hướng lãng phí thực phẩm.
8. **Admin Module (Quản trị):** Phục vụ công tác quản trị cấp cao (duyet các công thức nấu ăn do người dùng đóng góp, khóa/mở khóa tài khoản người dùng, quản lý danh mục thực phẩm hệ thống).

4.2 Cấp độ 2: Phân rã theo Vai Trò (Role-Based Modularity / Mô hình BCE)

Trong mỗi phân hệ, NaviMart áp dụng triệt để mô hình kiến trúc **BCE (Boundary - Control - Entity)** nhằm phân rã các lớp theo vai trò chức năng riêng biệt:

- **Tầng Boundary (Giao tiếp):** Chịu trách nhiệm tiếp nhận và phản hồi dữ liệu với bên ngoài. Trên Backend, vai trò này thuộc về các **Controller** (ví dụ: **AuthController**, **PantryController**) làm nhiệm vụ ánh xạ các HTTP requests và trả về HTTP responses thông qua các định dạng JSON/REST API thô, tuyệt đối không chứa logic nghiệp vụ hay truy vấn database trực tiếp.
- **Tầng Control (Điều khiển nghiệp vụ):** Đây là lõi xử lý của hệ thống, bao gồm các lớp **Service** (ví dụ: **AuthService**, **PantryService**). Tầng này chịu trách nhiệm thực thi các luồng xử lý và quy tắc nghiệp vụ phần mềm, độc lập hoàn toàn với cách hiển thị giao diện và chi tiết lưu trữ cơ sở dữ liệu.
- **Tầng Entity (Thực thể dữ liệu):** Đại diện cho cấu trúc dữ liệu cốt lõi và các thao tác tương tác trực tiếp với cơ sở dữ liệu (ví dụ: các lớp Entity của TypeORM như **UserEntity**, **FoodEntity** và các lớp Repository tương ứng). Tầng này chịu trách nhiệm đóng gói các tác vụ CRUD.

4.3 Cấp độ 3: Phân rã theo Trách Nhiệm (Functional Modularity)

Các lớp nghiệp vụ (Services) được chia nhỏ thành các phương thức (methods) đơn nhiệm, tập trung thực hiện đúng một nhiệm vụ nghiệp vụ duy nhất.

- **Ví dụ trong AuthService:** Các chức năng được phân tách cụ thể thành các phương thức độc lập:
 - **hashPassword(password):** Chỉ thực hiện nhiệm vụ băm mật khẩu bằng thuật toán bảo mật (bcrypt).
 - **generateToken(user):** Chỉ xử lý việc tạo mã JWT token từ thông tin người dùng.
 - **validateUser(credentials):** Chỉ làm nhiệm vụ kiểm tra và đối chiếu thông tin đăng nhập từ cơ sở dữ liệu.

Việc phân rã hàm siêu nhỏ này đảm bảo tính độc lập tuyệt đối. Nếu trong tương lai cần nâng cấp thuật toán mã hóa mật khẩu, lập trình viên chỉ cần chỉnh sửa nội dung bên trong hàm **hashPassword** mà không làm gián đoạn hay phải cấu trúc lại luồng **validateUser** hoặc tạo Token.

5 Đánh giá Coupling và Cohesion chi tiết từng Module của NaviMart

5.1 Đánh giá tổng quan hệ thống

- **Cohesion đạt mức Functional Cohesion (Cao nhất/Tốt nhất):** Nhờ áp dụng mô hình BCE, các lớp trong cùng một phân hệ hoạt động tập trung hoàn toàn vào một trách nhiệm nghiệp vụ duy nhất. Các Controller chỉ điều phối API, các Service chỉ lo nghiệp vụ cốt lõi, và các Entity chỉ lo tương tác cơ sở dữ liệu.
- **Coupling đạt mức Data Coupling (Thấp nhất/Tốt nhất):** Các module hoàn toàn không dùng chung các biến toàn cục (tránh Common Coupling). Không có module nào sửa đổi logic hay dữ liệu nội bộ của module khác (tránh Content Coupling). Giao tiếp hoàn toàn qua việc truyền tham số hoặc DTO đơn giản.

5.2 Module 1: Quản lý Xác thực và Người dùng (Auth Module)

- **Cohesion (Functional Cohesion):** Phân hệ Auth tập trung hoàn toàn vào mục tiêu xác thực và quản lý tài khoản người dùng. Các thành phần như `LoginView`, `RegisterView` chỉ làm nhiệm vụ thu thập thông tin; `AuthController` điều phối luồng đăng nhập; và `AuthService` phụ trách mã hóa mật khẩu cũng như xử lý token JWT.
- **Coupling (Data Coupling):** Module Auth tương tác với hệ thống gửi email OTP qua việc truyền các tham số nguyên thủy (primitive data) hoặc DTO cơ bản. Phân hệ này hoàn toàn độc lập và không chia sẻ bất kỳ biến toàn cục nào với các phân hệ khác như Family hay Pantry.

5.3 Module 2: Quản lý Nhóm Gia đình (Family Module)

- **Cohesion (Functional Cohesion):** Mọi chức năng liên quan đến việc tạo nhóm gia đình, rời nhóm, mời thành viên đều nằm trọn vẹn trong module Family. `FamilyController` không chứa bất kỳ logic nào về việc quản lý thực phẩm của nhóm (việc đó do Pantry đảm nhận), đảm bảo tính chuyên biệt nghiệp vụ ở mức tối đa.
- **Coupling (Data Coupling):** Module Family cung cấp dữ liệu cho Pantry và Shopping thông qua việc trả về tham số `groupId` dưới dạng kiểu dữ liệu nguyên thủy (Primitive Data Type). Tránh được tình trạng Common Coupling do Pantry không bao giờ truy cập trực tiếp vào các thuộc tính private của Family.

5.4 Module 3: Lập danh sách mua sắm (Shopping Lists Module)

- **Cohesion (Functional Cohesion):** Module tập trung giải quyết trọn vẹn luồng nghiệp vụ mua sắm (tạo danh sách, thêm món đồ cần mua, đánh dấu đã mua). Không có bất kỳ logic thừa nào lạc vào module này (ví dụ: việc cập nhật thực phẩm đã mua vào tủ lạnh sẽ được gọi sang module Pantry thay vì xử lý trực tiếp tại đây).
- **Coupling (Data Coupling):** Khi người dùng tick chọn “Đã mua”, `ShoppingController` truyền đối tượng `DTO_Item` sang `PantryController` để tủ lạnh tự động cộng số

lượng. Việc truyền nhận này thực hiện qua tham số rõ ràng, không can thiệp vào logic database nội bộ của Pantry.

5.5 Module 4: Quản lý Kho Thực phẩm (Pantry Module)

- **Cohesion (Functional Cohesion):** Pantry Module tập hợp tất cả các phương thức liên quan đến vòng đời thực phẩm trong nhà (thêm vào tủ, cập nhật số lượng, giảm số lượng khi nấu ăn, cảnh báo HSD). Tất cả các thao tác đều xoay quanh mục đích cốt lõi là quản lý kho thực phẩm.
- **Coupling (Data Coupling):** Pantry đóng vai trò trung tâm dữ liệu được nhiều module khác (như Meals, Shopping) gọi tới. Nó chỉ nhận/trả tham số danh sách nguyên liệu hoặc ID thực phẩm qua các API công khai chứ không chia sẻ chung bộ nhớ hay kết nối Database thô (tuân thủ nguyên lý ẩn giấu thông tin - Information Hiding).

5.6 Module 5: Kế hoạch ăn uống (Meals Module)

- **Cohesion (Functional Cohesion):** Module Meals đảm nhiệm chức năng lập lịch ăn uống (Meal Plan) và quản lý các phiên nấu ăn (Cooking Session). Mọi logic bên trong (chọn món, tính toán calo của bữa ăn) đều hướng tới một mục tiêu chung duy nhất là xây dựng bữa ăn hoàn chỉnh cho gia đình.
- **Coupling (Data Coupling):** Khi Meals cần trừ nguyên liệu trong tủ lạnh sau khi nấu, nó gọi phương thức `deductIngredients(list)` của `PantryController`. Khi cần lấy công thức nấu ăn, nó nhận một đối tượng `RecipeDTO` từ module Recipes chứ không tự ý truy cập cơ sở dữ liệu của phân hệ khác.

5.7 Module 6: Công thức & Trợ lý ảo (Recipes & AI Chef Module)

- **Cohesion (Functional Cohesion):** Tập trung hoàn toàn vào việc lưu trữ, tìm kiếm và hiển thị công thức nấu ăn. Trợ lý thông minh AI Chef (tính năng phức tạp) cũng được tích hợp vào đây vì bản chất nó phục vụ cho việc tạo hoặc giải đáp các thắc mắc xoay quanh chủ đề công thức món ăn, tránh phân tán mã nguồn.
- **Coupling (Data Coupling):** Khi tương tác với External API của OpenAI/Gemini để phân tích nguyên liệu, AI Chef chỉ truyền các tham số cần thiết (danh sách nguyên liệu) dưới định dạng chuỗi JSON thô, tạo ra sự liên kết lỏng lẻo lý tưởng với dịch vụ của bên thứ ba.

5.8 Module 7: Thống kê & Báo cáo (Statistics Module)

- **Cohesion (Functional Cohesion):** Phân hệ này tập hợp tất cả các hàm logic cần thiết để tạo ra các báo cáo trực quan cho Dashboard và biểu đồ, cùng phục vụ mục tiêu duy nhất là hiển thị số liệu phân tích chi tiêu và lãng phí thực phẩm cho người dùng.

- **Coupling (Data Coupling):** Statistics Module đóng vai trò Read-only đối với toàn bộ hệ thống. Nó truyền các đối tượng DTO hoặc ID (như `userId`, `dateRange`) tới các Controller của Pantry hoặc Shopping để lấy số liệu, tuyệt đối không can thiệp chỉnh sửa dữ liệu gốc.

5.9 Module 8: Quản trị hệ thống (Admin Module)

- **Cohesion (Functional Cohesion):** Thực hiện toàn bộ các tác vụ quản trị hệ thống của Admin (duyet công thức, khóa/mở khóa tài khoản, xem thống kê hệ thống). Module này cô lập các tác vụ rủi ro cao vào chung một nơi để dễ dàng kiểm soát quyền truy cập.
- **Coupling (Data Coupling):** Khi khóa tài khoản của người dùng, Admin gọi đến API nội bộ của module Auth và truyền tham số `UserId`. Nó không dùng chung kết nối database hay tác động trực tiếp vào vùng nhớ của module Auth.

6 Đánh giá nguyên lý SOLID chi tiết từng Module của NaviMart

6.1 Đánh giá tổng quan hệ thống

Hệ thống NaviMart được thiết kế bám sát 5 nguyên lý SOLID: chia nhỏ kiến trúc theo mô hình BCE để đảm bảo đơn trách nhiệm (SRP), sử dụng giao diện trừu tượng và các Design Pattern để mở rộng dễ dàng (OCP/DIP), hạn chế tối đa việc kế thừa lớp cụ thể để tránh lỗi runtime (LSP), và chia nhỏ các giao diện công kênh để tránh phụ thuộc thừa (ISP).

6.2 Module 1: Quản lý Xác thực và Người dùng (Auth Module)

- **SRP (Single Responsibility):** Các lớp trong module có một lý do duy nhất để thay đổi. `AuthService` chỉ thay đổi khi thuật toán băm mật khẩu hoặc định dạng token JWT thay đổi. Nó không chứa mã nguồn hiển thị giao diện hay truy cập DB trực tiếp.
- **OCP (Open-Closed):** Lớp xác thực mở rộng dễ dàng. Khi cần thêm phương thức đăng nhập bằng Google (OAuth2), ta chỉ cần viết thêm lớp `GoogleAuthStrategy` thực thi giao diện `IAuthStrategy` mà không cần thay đổi logic xác thực email/password sẵn có.
- **LSP (Liskov Substitution):** Mọi hình thức xác thực dù bằng JWT hay Session đều có thể thay thế vào interface `ITokenProvider` mà không làm thay đổi hành vi mong đợi của hệ thống.
- **ISP (Interface Segregation):** Các API xác thực được tách riêng biệt: `IUserAuthentication` (cho Login/Register) và `IUserProfile` (cho cập nhật thông tin). Tránh tạo ra một interface công kênh chứa cả hai nhóm hàm.

- **DIP (Dependency Inversion):** `AuthController` phụ thuộc vào Interface `IAuthService` thay vì phụ thuộc vào lớp triển khai cụ thể `AuthServiceImpl`. Điều này giúp dễ dàng mock dữ liệu khi viết Unit Test cho Controller.

6.3 Module 2: Quản lý Nhóm Gia đình (Family Module)

- **SRP (Single Responsibility):** `FamilyService` chỉ lo các nghiệp vụ liên kết tài khoản gia đình. Việc gửi thông báo mời thành viên tham gia nhóm được chuyển giao cho `NotificationService` xử lý.
- **OCP (Open-Closed):** Khi cần mở rộng các quy tắc giới hạn thành viên trong gia đình (ví dụ: tài khoản thường tối đa 5 người, tài khoản Premium không giới hạn), ta có thể tích hợp mẫu thiết kế Strategy (`IFamilyLimitStrategy`) mà không cần sửa đổi mã nguồn gốc của `FamilyService`.
- **LSP (Liskov Substitution):** Hai vai trò trong gia đình là `FamilyOwner` (chủ nhóm) và `FamilyMember` (thành viên) đều kế thừa từ lớp cơ sở và có thể thay thế cho nhau trong các hàm kiểm tra tính hợp lệ của nhóm gia đình mà không gây lỗi logic.
- **ISP (Interface Segregation):** Thay vì sử dụng một interface lớn `IFamilyManager`, hệ thống chia nhỏ thành `IFamilyReader` (chỉ xem thành viên) dành cho người dùng thông thường và `IFamilyWriter` (chỉnh sửa nhóm) dành cho Admin của nhóm gia đình.
- **DIP (Dependency Inversion):** `FamilyController` không phụ thuộc trực tiếp vào `FamilyEntity` mà gọi gián tiếp thông qua interface `IFamilyService`, đảm bảo module cấp cao không phụ thuộc vào chi tiết triển khai ở cấp thấp.

6.4 Module 3: Lập danh sách mua sắm (Shopping Lists Module)

- **SRP (Single Responsibility):** Lớp `ShoppingListEntity` chỉ đảm nhận việc lưu trữ thông tin tên danh sách mua sắm. Các mặt hàng cụ thể bên trong danh sách được tách ra thành một thực thể riêng biệt là `ShoppingItemEntity`. Mỗi class quản lý một bảng duy nhất trong database.
- **OCP (Open-Closed):** Logic gợi ý mua sắm thông minh (tự động gợi ý các nguyên liệu còn thiếu) được đóng gói trong interface `ISuggestionStrategy`. Nếu muốn đổi thuật toán gợi ý từ “gợi ý theo thực đơn” sang “gợi ý theo tần suất mua”, ta chỉ cần tạo một Strategy mới implement interface này.
- **LSP (Liskov Substitution):** Tất cả các loại danh sách mua sắm (Danh sách thông thường, Danh sách sự kiện, Danh sách tự động tạo) đều triển khai interface `IShoppingList` và hoạt động hoàn hảo khi được truyền vào hàm `renderList()`.
- **ISP (Interface Segregation):** Giao diện `IShoppingItemManager` được chia nhỏ thành các interface chuyên biệt như `IAddItem` (chỉ thêm) và `ICheckItem` (chỉ đánh dấu đã mua) để các màn hình UI thu gọn không cần phải nhận toàn bộ các method không cần thiết.

- **DIP (Dependency Inversion):** `ShoppingController` không phụ thuộc trực tiếp vào `PantryController` mà phụ thuộc vào interface `IPantryService` khi thực hiện tác vụ tự động chuyển thực phẩm đã mua vào tủ lạnh.

6.5 Module 4: Quản lý Kho Thực phẩm (Pantry Module)

- **SRP (Single Responsibility):** Lớp `PantryService` chỉ chịu trách nhiệm xử lý logic quản lý số lượng tồn kho của thực phẩm. Việc phát các cảnh báo (Notification) khi thực phẩm sắp hết hạn sử dụng được chuyển giao hoàn toàn sang lớp `NotificationService`.
- **OCP (Open-Closed):** Dễ dàng mở rộng phương thức nhập dữ liệu thực phẩm (ví dụ: quét mã vạch Barcode hoặc dùng AI nhận diện hóa đơn). Lập trình viên chỉ cần thêm lớp `BarcodePantryInput` triển khai interface `IPantryInput` mà không cần chỉnh sửa luồng nhập thủ công `ManualPantryInput`.
- **LSP (Liskov Substitution):** Mọi nguồn dữ liệu thực phẩm (do người dùng tự nhập tay hoặc do hệ thống tự động quét từ hóa đơn) đều được xử lý một cách chính xác và đồng bộ bởi Pantry Module mà không gây ra lỗi xung đột kiểu dữ liệu.
- **ISP (Interface Segregation):** Tách biệt giao diện công kênh thành các giao diện nhỏ hơn: `IExpiryChecker` (chỉ phục vụ các tác vụ chạy ngầm quét thực phẩm hết hạn hàng ngày) và `IPantryManager` (phục vụ các thao tác CRUD của người dùng).
- **DIP (Dependency Inversion):** Các hàm trừ số lượng nguyên liệu thực phẩm trong Pantry không thực hiện kết nối cơ sở dữ liệu trực tiếp ở cấp thấp, mà gọi thông qua interface trừu tượng `IPantryRepository`.

6.6 Module 5: Kế hoạch ăn uống (Meals Module)

- **SRP (Single Responsibility):** Lớp `MealController` chỉ quản lý logic sắp xếp các bữa ăn vào các khung giờ trong ngày. Việc trừ nguyên liệu thực phẩm khi nấu ăn hoàn tất được chuyển giao sang cho `PantryController` xử lý.
- **OCP (Open-Closed):** Cho phép dễ dàng mở rộng và thay đổi các tiêu chí gợi ý thực đơn (Meal Suggestion). Module cho phép truyền các thuật toán `IMealSuggestStrategy` khác nhau (như gợi ý theo lượng Calo mong muốn, hoặc gợi ý dựa trên thực phẩm sắp hết hạn) mà không cần chỉnh sửa mã nguồn của Controller.
- **LSP (Liskov Substitution):** Mọi loại phiên bữa ăn (Bữa sáng, Bữa trưa, Bữa tối, Ăn vặt) kế thừa từ lớp cơ sở `AbstractMealSession` đều có thể thay thế hoàn toàn cho nhau trong phương thức tính toán tổng lượng Calo hàng ngày của gia đình.
- **ISP (Interface Segregation):** Chia nhỏ giao diện `IMealService` thành hai phần chuyên biệt: `IMealPlanner` (dành cho người dùng lập lịch ăn) và `IMealExecutor` (dành cho luồng xác nhận nấu hoàn thành và tự động trừ kho).
- **DIP (Dependency Inversion):** Các phân hệ khác muốn tương tác với Meals đều phải thông qua giao diện trừu tượng `IMealService`, tuyệt đối không được khởi tạo trực tiếp instance của lớp triển khai `MealServiceImpl`.

6.7 Module 6: Công thức & Trợ lý ảo (Recipes & AI Chef Module)

- **SRP (Single Responsibility):** Tác vụ kết nối và gọi API bên ngoài của OpenAI được cô lập hoàn toàn trong lớp dịch vụ `OpenAiService`. Controller của Recipes chỉ làm nhiệm vụ định tuyến (routing) request. Lớp thực thể `RecipeEntity` chỉ lưu trữ dữ liệu công thức, không chứa logic phân tích cú pháp (parse) dữ liệu JSON trả về từ AI.
- **OCP (Open-Closed):** Thiết kế xuất sắc nhờ định nghĩa giao diện `IAiService`. Nếu tương lai NaviMart muốn chuyển đổi từ ChatGPT sang Google Gemini hoặc một mô hình ngôn ngữ lớn chạy cục bộ (Llama), lập trình viên chỉ cần tạo lớp mới triển khai interface này mà không cần sửa đổi bất kỳ dòng mã nào trong `AiChefController`.
- **LSP (Liskov Substitution):** Mọi mô hình AI được tích hợp vào `IAiService` đều đảm bảo trả về định dạng đầu ra chuẩn là đối tượng `RecipeDTO`, giữ vững tính đúng đắn và kỳ vọng của các module tiêu thụ dữ liệu này.
- **ISP (Interface Segregation):** Giao diện `IRecipeManager` được phân tách rõ ràng thành các tính năng công khai (xem công thức, tìm kiếm) dành cho tất cả người dùng và các tính năng nội bộ (thêm, sửa, xóa) dành riêng cho tác giả công thức, tránh cấp thừa quyền sửa đổi cho các client chỉ đọc.
- **DIP (Dependency Inversion):** Phân hệ phụ thuộc hoàn toàn vào giao diện trừu tượng `IAiService` thay vì phụ thuộc trực tiếp vào lớp cụ thể `OpenAiServiceImpl`, giúp việc thay đổi thư viện AI trở nên cực kỳ dễ dàng.

6.8 Module 7: Thống kê & Báo cáo (Statistics Module)

- **SRP (Single Responsibility):** Việc tính toán và xử lý số liệu báo cáo được giao hoàn toàn cho `ReportService`, trong khi việc kết xuất đồ họa (render biểu đồ) thuộc trách nhiệm riêng biệt của `DashboardView`. Khi thay đổi thư viện vẽ biểu đồ, ta chỉ cần chỉnh sửa lớp View mà không tác động tới logic tính toán của Service.
- **OCP (Open-Closed):** Có thể dễ dàng tích hợp thêm các loại báo cáo mới (ví dụ: báo cáo xu hướng nấu ăn) bằng cách viết thêm lớp mới `TrendReportStrategy` triển khai từ interface `IReportStrategy` mà không phải chỉnh sửa code hiện tại trong `ReportService`.
- **LSP (Liskov Substitution):** Mọi lớp báo cáo đều triển khai giao diện chung `IReport`. Lập trình viên có thể thay thế `WasteReport` bằng `SpendingReport` ở luồng xuất file PDF báo cáo mà không gây ra lỗi cấu trúc chương trình.
- **ISP (Interface Segregation):** Giao diện xuất dữ liệu `IExportable` (phục vụ tải file Excel/PDF) được tách riêng biệt. Các báo cáo không hỗ trợ tải về máy sẽ không bị ép buộc phải triển khai interface này.
- **DIP (Dependency Inversion):** Các module khác khi có nhu cầu ghi nhận số liệu thống kê sẽ gọi thông qua interface trừu tượng `IStatsLogger` thay vì gọi trực tiếp tới lớp cụ thể.

6.9 Module 8: Quản trị hệ thống (Admin Module)

- **SRP (Single Responsibility):** Lớp ranh giới `AdminBoundary` chỉ tập trung lo giao diện quản trị hệ thống. Mọi logic xác định điều kiện tài khoản có được phép khóa hay không được đóng gói hoàn toàn trong lớp `AdminService`.
- **OCP (Open-Closed):** Dễ dàng mở rộng các tính năng quản trị. Khi cần thêm nghiệp vụ quản lý mới như “Duyệt bình luận công thức”, ta chỉ cần viết thêm phương thức nghiệp vụ mới trong `AdminService` mà không làm ảnh hưởng hay phá hỏng luồng duyệt công thức sẵn có.
- **LSP (Liskov Substitution):** Bất kỳ loại thực thể nào bị người dùng báo cáo (như `Recipe`, `Comment`, `User`) đều triển khai chung giao diện `IReportable`. Nhờ đó, `AdminService` có thể xử lý báo cáo hàng loạt mà không cần sử dụng các câu lệnh kiểm tra kiểu dữ liệu thủ công như `instanceof`.
- **ISP (Interface Segregation):** Không sử dụng một interface chung lớn `ISystemManager`. Hệ thống tách biệt thành interface `IUserModerator` (dành cho người kiểm duyệt quản lý tài khoản) và `IRecipeModerator` (dành cho người duyệt công thức nấu ăn).
- **DIP (Dependency Inversion):** Module Admin phụ thuộc hoàn toàn vào các giao diện trừu tượng của các module khác. Ví dụ, khi cần lấy danh sách tài khoản người dùng để hiển thị trên màn hình quản trị, Admin gọi interface `IUserReaderService` thay vì gọi trực tiếp vào thực thể `UserEntity`.

7 Kết luận

Kiến trúc của hệ thống NaviMart đã được xây dựng và thiết kế tuân thủ nghiêm ngặt các tiêu chuẩn chất lượng cao về Công nghệ phần mềm:

- Đạt được mục tiêu **Loose Coupling** (liên kết lỏng lẻo) và **Tight Cohesion** (gắn kết chặt chẽ) ở cấp độ lý tưởng nhất (Data Coupling & Functional Cohesion) trên cả 8 phân hệ cốt lõi.
- Áp dụng triệt để 5 nguyên tắc thiết kế **SOLID** tạo nên nền tảng mã nguồn vững chắc, sạch sẽ, dễ dàng viết Unit Test độc lập và sẵn sàng đáp ứng nhanh chóng các yêu cầu mở rộng tính năng cũng như thay đổi công nghệ trong tương lai của hệ thống.